

SDR-SLC

SDR-SLC Protocol Suite Architecture

Field	Value
Document	SDR-SLC-PS-1.0
Title	SDR-SLC Protocol Suite Architecture
Version	1.0 (Protocol) Release 1.1 (revised for implementers)
Status	RELEASE CANDIDATE
Date	May 2026
Author	Ivo van Ling, PA2IX
Organisation	IXChange BV
Licence	Creative Commons CC BY 4.0 — Open Source

Table of Contents

Table of Contents	2
Revision History.....	4
1. Normative Language	5
2. Philosophy and Design Goals	6
2.1 Why SDR-SLC Exists	6
2.2 The Core Architectural Philosophy: Push DSP to the Host	6
2.3 How This Differs from Server-Centric SDR Architectures.....	7
2.4 Why This Is Ideal for HAM Radio Experimenters.....	8
2.5 The Target Hardware Cost	9
2.6 What You Get When It Works.....	9
3. Scope	11
4. Terminology.....	12
5. Document Set.....	13
6. The Slice Model: Wideband Capture, Host-Side Processing	14
6.1 What the Hardware Does.....	14
6.2 What the Client Does	14
6.3 What This Means for Per-Slice Metering	14
6.4 What This Means for Waterfall and Spectrum Display	14
7. System Architecture	16
7.1 Protocol Planes.....	16
7.2 Startup Dependency Order	16
7.3 System Architecture Diagram	17
8. Network Topology.....	18
8.1 Local Network Operation	18
8.2 IP Addressing.....	18
8.3 Multiple Devices.....	18
8.4 Managed Switches and IGMP	18
9. Time Synchronisation	19
9.1 Role of PTPv2.....	19
9.2 Clock Source Hierarchy	19
10. I/Q Format and Sample Depth.....	20
11. Conformance	21
11.1 Device Conformance by Profile	21
11.2 Client Conformance	22
12. Typical Session Flow.....	23
13. Design Rationale	24
13.1 Why mDNS/DNS-SD?	24

13.2 Why JSON/TCP?	24
13.3 Why VITA-49?.....	24
13.4 Why Host-Side DSP?.....	24
13.5 Why 64-bit Frequency Fields?	24
14. Hardware Reference Design	25
14.1 The Tayloe Detector	25
14.2 Microcontroller	25
14.3 ADC Interface	25
14.4 A Minimal Working Build	25
15. Security Considerations.....	27
16. Forward Compatibility.....	28
17. References	29
17.1 Normative.....	29
17.2 Informative	29
Appendix A Document Identifier Register.....	30

Revision History

Rev	Date	Author	Description
1.0	2026-05-01	I. van Ling	Initial public draft.
1.1	2026-05-23	I. van Ling	Revised for implementers. Added §2 Philosophy and Design Goals with host-offload rationale, HAM builder narrative, hardware cost discussion, and architectural comparison with server-centric SDR systems. Added §6 Slice Model explanation. Updated conformance tables to reflect four-profile model from SDR-SLC-CP-1.0-v2. Added §14 Hardware Reference Design. Aligned editorial style with SDR-SLC-CP-1.0-v2 and SDR-SLC-VITA-1.0-v2.

1. Normative Language

The key words SHALL, SHALL NOT, SHOULD, SHOULD NOT, and MAY are to be interpreted as described in RFC 2119.

Keyword	Interpretation
SHALL	Absolute requirement. Non-conformant if omitted.
SHALL NOT	Absolute prohibition.
SHOULD	Strong recommendation. Deviation requires justification.
MAY	Optional. Implementations may include or omit freely.

2. Philosophy and Design Goals

2.1 Why SDR-SLC Exists

Software-defined radio has reached a level of technical maturity where the barriers are no longer theoretical. The theory is well understood, the algorithms are published, the hardware components are cheap. The remaining barriers are practical: cost, platform exclusivity, and closed interfaces. SDR-SLC is a direct response to all three.

Modern networked SDR transceivers represent a genuine architectural advance: I/Q samples flow over Ethernet, signal processing runs on a connected computer, and the hardware itself is kept relatively simple. The approach is sound and has proven itself in professional and serious-amateur use. The problem is not the architecture. The problem is what surrounds it in the products that implement it.

Cost is the first barrier. High-end SDR transceivers from established manufacturers are priced for the professional and the serious contester. The experimenter on a modest budget, the student learning the craft, the HAM operator who wants to understand what is inside their radio rather than just use it — these people are priced out of the market before they start. A receiver-only board that costs more than a used car is not accessible hardware.

Platform exclusivity is the second. The client software that accompanies most high-end networked SDR hardware is written for Windows. An engineer who has spent a career in telecommunications and has used Apple hardware for two decades should not be required to maintain a Windows installation to operate a radio. macOS support, where it exists at all, tends to be a community afterthought rather than a first-class commitment from the manufacturer.

Proprietary interfaces are the third. Control protocols and streaming formats are frequently documented only to the extent required for third-party plugin authors, if at all. Implementing a fully conformant client from first principles — without access to a reference implementation or a complete specification — is impractical for the individual experimenter. The protocol becomes the exclusive property of the manufacturer rather than a shared foundation for the community.

NOTE

SDR-SLC (Super Low Cost / Slice) is a response to these three constraints. The dual name is intentional: the hardware target is Super Low Cost by design, and the software model is built around independently tunable slices within a wide captured bandwidth.

2.2 The Core Architectural Philosophy: Push DSP to the Host

The fundamental architectural decision in SDR-SLC is the most important one, and it is worth stating clearly: the radio hardware does as little as possible. The host computer does everything else.

The radio hardware has three responsibilities: capture I/Q samples from the antenna, convert them to digital form, and stream them over Ethernet. That is it. It does not demodulate. It does not filter. It does not compute FFTs. It does not run a spectrum analyser. It does not decide which frequency you are listening to within the captured bandwidth. It sends raw wideband I/Q samples and gets out of the way.

Everything that requires computation — the digital downconversion that extracts a specific slice from the wideband capture, the filtering that sets the passband, the demodulation that recovers the audio,

the FFT that drives the waterfall display, the noise blanking, the automatic gain control, the squelch — all of it runs on the host computer. On a MacBook with Apple Silicon, this is trivial. The M-series chips have dedicated signal processing pathways that can handle multiple simultaneous slices, full-bandwidth FFT for the waterfall display, and audio output while barely warming up.

This is not a compromise. It is a deliberate choice that produces a better system in every dimension that matters for the target use case:

- The hardware is simpler, cheaper, and easier to build. An ADC, a microcontroller, and an Ethernet interface are all you need. No FPGA. No DSP chip. No complex firmware for signal processing.
- The DSP is more capable, not less. A MacBook runs more compute than any embedded FPGA that would fit in a €100 budget. The client can run more slices, wider FFTs, and more sophisticated algorithms than any fixed hardware implementation.
- The DSP is completely software-defined in the truest sense. Changing the demodulation algorithm, adding a new digital mode, improving the noise blanking — none of these require hardware changes or firmware updates. They are Swift code changes in the client.
- The S-meter and other per-slice measurements are computed in the client from the slice I/Q, where all the relevant context — passband width, demodulation mode, frequency offset — is known. They are more accurate there than they could be on the hardware, which has no knowledge of what the client is doing with the samples.

2.3 How This Differs from Server-Centric SDR Architectures

Some networked SDR systems place the signal processing on the radio hardware. The radio contains a powerful FPGA or DSP that performs digital downconversion, channelisation, and spectrum analysis internally. The client receives pre-processed slices — already decimated to a narrow bandwidth around the station of interest — and pre-computed FFT data for the waterfall display.

This is a legitimate architectural choice and it works well for their target use case. It enables thin clients: a hardware panel with a small display and no GPU can show a waterfall because the radio sends pre-computed display data. It enables per-slice metering on the radio side because the radio's DSP knows exactly which slice it is processing. It enables lower network bandwidth for the data path because narrow slice I/Q is smaller than wideband I/Q.

But it comes at a cost: the signal processing FPGA drives up the hardware price dramatically. It also fixes the DSP algorithms in hardware. The radio does the FFT on the FPGA, so the FFT parameters — size, windowing, frame rate — are hardware constraints, not software choices. Adding a new digital mode requires a firmware update from the manufacturer. The architecture assumes a relatively dumb client and a smart radio.

SDR-SLC inverts this assumption. The radio is deliberately dumb. The client is deliberately smart. This is the right trade-off when the client is a MacBook and the target user is a HAM radio experimenter who wants to write their own DSP code, not just use what the manufacturer provides.

Concern	Server-centric SDR	SDR-SLC
Signal processing location	Radio hardware (FPGA/DSP)	Host computer (CPU/GPU)

Concern	Server-centric SDR	SDR-SLC
Per-slice metering	On the radio (knows each slice)	On the client (knows each slice)
Waterfall/FFT	Pre-computed on radio, streamed	Computed on host from wideband I/Q
Hardware cost	High (FPGA required)	Low (ADC + MCU + Ethernet)
DSP flexibility	Fixed by firmware	Fully programmable in client software
Platform requirements	Often Windows-centric	macOS primary, platform-agnostic protocol
New digital modes	Firmware update required	Client software update only
Network data rate	Low (narrow slice I/Q)	Higher (wideband I/Q)
Home builder feasibility	No (FPGA expertise required)	Yes (STM32 + Tayloe detector)

NOTE

The higher network data rate is not a problem in practice. A 2 MHz wideband stream at 16-bit I/Q requires approximately 64 Mbit/s. A standard home Ethernet switch handles this trivially. The 1 Gbit/s link on a modern MacBook has 15× the available bandwidth.

2.4 Why This Is Ideal for HAM Radio Experimenters

The HAM radio community has a long tradition of home construction. Building your own equipment, understanding how it works at the component level, and adapting it to your specific needs is a core part of the hobby for many operators. SDR-SLC is designed to support this tradition at the intersection of modern RF technology and software-defined radio.

The hardware side is genuinely buildable by a competent HAM with basic electronics skills. The Tayloe detector — the quadrature sampling detector used in the IXChange reference design — is an elegantly simple circuit. Four bilateral switches, two op-amps, and a clock source. It was invented by Dan Tayloe (N7VE) and described in QEX magazine. It is already documented in the amateur radio literature, and builders have constructed it on veroboard with point-to-point wiring. The I/Q output from the Tayloe detector feeds directly into the ADC inputs on the microcontroller.

The microcontroller side is equally accessible. The STM32H7 family is well supported by open-source toolchains, has a large and active community, and costs under €15 in single quantities. Every peripheral needed for SDR-SLC — SPI for the ADC, Ethernet MAC, DMA for continuous sample streaming, a hardware timer for packet pacing — is built into the chip. The lwIP TCP/IP stack is free, well-documented, and runs comfortably on the H7.

The software side on the host is taken care of. The SDR-SLC Swift client runs on macOS and handles all the DSP. The HAM builder's job is the hardware and the firmware. A working SDR receiver is achievable in a weekend of building and a week of firmware development. A transceiver with PTT switching adds another weekend on the hardware side and modest additional firmware work.

TIP

You do not need to build the IXChange reference design to get started. An RTL-SDR dongle (€25) bridged to SDR-SLC via a daemon running on a Raspberry Pi (€40) is a fully conformant

Tier 1 implementation. The protocol is hardware-agnostic. Start with the RTL-SDR bridge, learn the protocol, then build your own hardware when you are ready.

2.5 The Target Hardware Cost

A complete SDR-SLC RX-only implementation, from antenna connector to Ethernet port, should cost under €50 in components. A full transceiver implementation including a simple PA for QRP operation should cost under €100. These are targets for a home-built board; a pre-assembled board from a small producer should still come in under €100 and be accessible to the average amateur.

Component	Example	Approx. cost
Microcontroller	STM32H743 or similar H7 series	€10–15
Ethernet PHY	LAN8720A or integrated	€3–5
Quadrature clock source	SI5351A (I2C synthesiser)	€3–5
Taylor detector switches	74HC4066 or FST3253	€2–3
I/Q buffer amplifiers	NE5532 or OPA2134	€2–3
Band-pass / low-pass filters	LC filter, plug-in modules	€5–10
PCB (JLCPCB, 5 boards)	2-layer, ~10 cm × 8 cm	€5–10
Passives, connectors, power regulation	Various	€5–10
Total (RX only)		~€35–60
PA module for TX (QRP, 5W)	IRF510 or similar	€10–20
T/R relay and driver	Reed relay or solid state	€5–10
Total (RX/TX transceiver)		€50–90

These cost figures assume component purchases from the usual sources (Mouser, DigiKey, AliExpress, or the shack junk box). An experienced builder with a well-stocked junk box can build the receiver section for almost nothing. The SI5351A is the most critical component — everything else is standard analogue circuitry that has been in HAM shacks for decades.

WARNING

The firmware is not trivial. Streaming I/Q samples at 1-2 MSPS continuously over Ethernet, assembling VITA-49 packets, managing TCP and UDP simultaneously, and computing fractional timestamps requires careful firmware design. Allow several weekends for the firmware if you are new to STM32 development. The lwIP integration in particular has subtleties around DMA buffer management. But it is achievable, and there is an active community to help.

2.6 What You Get When It Works

When a working SDR-SLC device is connected to the network and the client is running, the experience is that of a high-quality software-defined receiver or transceiver, controlled from macOS with a modern native interface. The waterfall shows the full captured bandwidth — 1 or 2 MHz of

spectrum, depending on the sample rate. Multiple simultaneous receive slices can be positioned anywhere in that bandwidth. Each slice demodulates independently with its own mode, passband, and audio output. The S-meter updates at the rate the client DSP runs, which is fast enough to track even rapid QSB fading.

For a transceiver implementation, pushing PTT switches the radio to transmit. The client generates the I/Q baseband audio, packages it as VITA-49 UDP packets, and sends it to the radio hardware. The hardware converts it to RF and drives the PA. The turnaround time is determined by network latency and audio buffer size — on a local Ethernet switch, this is fast enough for normal SSB and CW operation.

For the builder, the gratification is that the complicated part — the part that took entire product generations at established manufacturers to get right — is running on your MacBook where you can inspect it, modify it, and improve it. The hardware is the part you built. The software is open. The protocol is documented. Nothing is a black box.

3. Scope

This document is the primary architectural reference for the SDR-SLC protocol suite. It specifies:

- The design philosophy and architectural goals of the suite (covered in §2 above).
- The three protocol planes (SLC-DISC, SLC-CP, VITA-SLC) and their roles.
- The relationships and dependencies between protocols.
- Network topology, IP addressing, and multicast requirements.
- The I/Q stream and slice model and how wideband capture differs from per-slice streaming.
- The role of PTPv2 time synchronisation.
- The I/Q format and sample-depth negotiation model.
- Conformance requirements for hardware and client implementations.
- Hardware reference design guidance.

NOTE

Protocol-level command details, packet formats, and field definitions are in the individual specifications. Read this document first for context, then read the sub-specifications for implementation details.

4. Terminology

Term	Definition
SDR-SLC Client	Host software on macOS, Linux, or Windows that connects to SDR-SLC hardware and performs all signal processing.
SDR-SLC Device	Radio hardware that captures I/Q samples and streams them. Contains ADC, microcontroller, and Ethernet interface. No signal processing.
SLC-DISC	Service discovery layer. mDNS/DNS-SD advertisement (RFC 6762/6763). How clients find devices on the LAN.
SLC-CP	Control plane. JSON/TCP protocol for radio configuration and stream session management.
VITA-SLC	I/Q stream transport. VITA-49 constrained profile for UDP sample streaming.
VITA-A/1.0	The SDR-SLC constrained IF Data packet profile of VITA-49. Defined in SDR-SLC-VITA-1.0.
VITA-A/1.0-CTX	The SDR-SLC constrained Context packet profile. Carries signal type and Reference Level metadata.
Slice	A client-side receiver defined by a centre frequency and passband width, derived from the wideband I/Q stream by digital downconversion (DDC). There is no hardware concept of a slice — it is entirely a client DSP abstraction.
Wideband stream	The full captured bandwidth from the hardware ADC, streamed to the client. Typically 1–2 MHz for HF SDR operation.
DDC	Digital Downconverter. Client-side signal processing that extracts a narrow-band slice from the wideband stream. Consists of a frequency translation, low-pass filter, and decimator.
Direction	Either rx (receive) or tx (transmit).
Stream ID	Opaque identifier assigned by the device to a VITA-49 UDP stream session.
Reference Level	Power at the antenna port corresponding to a full-scale I/Q sample, in dBm. Used for calibrated power display.
OUI	Organizationally Unique Identifier. IXChange BV: 0x402814 (40-28-14).
Taylor detector	A quadrature sampling detector invented by Dan Taylor (N7VE). Four bilateral switches driven by a quadrature clock sample the incoming RF signal directly. Simple, low-component-count, excellent dynamic range.
mDNS	Multicast DNS (RFC 6762). Resolves .local. hostnames without a central DNS server. Used by SLC-DISC.
DNS-SD	DNS-Based Service Discovery (RFC 6763). Advertises services by type over mDNS. Used by SLC-DISC.

5. Document Set

The SDR-SLC Protocol Suite consists of four documents. This document is the primary architectural reference. Read it first.

Document ID	Title	Purpose	Read order
SDR-SLC-PS-1.0	Protocol Suite Architecture	Philosophy, architecture, relationships, conformance summary, hardware guidance.	1 — Read first
SDR-SLC-DISC-1.0	SLC-DISC Service Discovery	mDNS/DNS-SD advertisement, TXT record fields, IGMP, verification procedures.	2
SDR-SLC-CP-1.0	SLC-CP Control Plane Specification	JSON/TCP control protocol. All commands, full implementation guide, conformance.	3
SDR-SLC-VITA-1.0	VITA-SLC I/Q Stream Transport	VITA-49 packet format, timestamps, Reference Level, implementation walkthrough.	4

6. The Slice Model: Wideband Capture, Host-Side Processing

The slice model is the architectural concept that distinguishes SDR-SLC from hardware-centric SDR systems and that justifies the low hardware cost. Understanding it is essential before reading the protocol specifications.

6.1 What the Hardware Does

The SDR-SLC hardware captures a wide swath of spectrum from the antenna. With a 2 MSPS ADC, it captures a 2 MHz slice of the HF spectrum centred on whatever frequency the local oscillator is tuned to. With a 1 MSPS ADC, the captured bandwidth is 1 MHz. This captured signal is converted to digital I/Q samples and streamed over Ethernet to the host computer. Every sample of every frequency within that 1–2 MHz band is present in the stream.

The hardware has no knowledge of what is in that spectrum. It does not know you are listening to an SSB station at 14.205 MHz. It does not know the difference between a CW signal at 7.030 MHz and a digital mode signal at 7.047 MHz. It simply captures everything and streams it. This is intentional.

6.2 What the Client Does

The client receives the wideband I/Q stream and creates software-defined ‘slices’ within it. A slice is defined by a centre frequency and a passband width. To listen to a USB station at 14.205 MHz when the hardware is centred on 14.1 MHz, the client creates a slice centred 105 kHz above the hardware LO. The DDC in the client downconverts that part of the wideband spectrum to baseband, applies a low-pass filter with the appropriate passband for USB (typically 2.4 kHz), and passes the result to the USB demodulator which recovers audio.

The same wideband stream can simultaneously support multiple slices. While listening to USB at 14.205 MHz, you can create a second slice at 14.225 MHz and listen to a second station, and create a third slice at 14.175 MHz just to monitor. All three slices run simultaneously from the same hardware stream. The hardware does not know or care. It is still streaming the same 2 MHz of spectrum.

6.3 What This Means for Per-Slice Metering

Because slices are entirely client-side, the S-meter for each slice must also be computed client-side. The hardware cannot compute a per-slice S-meter because it does not know which slices exist. The client, however, knows exactly what it needs: after DDC and filtering, the RMS amplitude of the decimated slice I/Q directly gives the signal strength in dBFS. Combined with the Reference Level from the VITA-49 Context packet, this converts to absolute power in dBm at the antenna port.

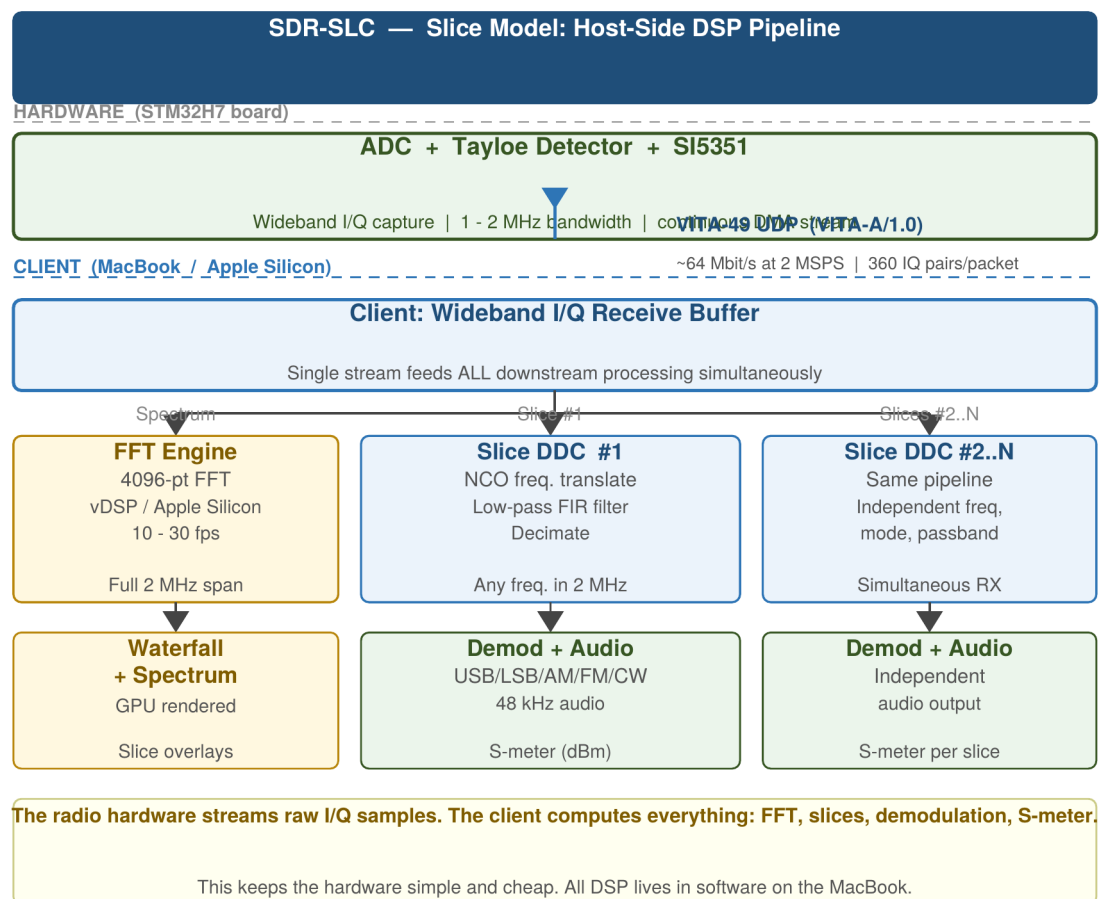
This is not a limitation. It is actually more accurate than per-slice metering on the hardware side, because the client applies the measurement to the filtered slice output rather than the raw wideband input. The measurement is made after the passband filter removes out-of-band noise, which is physically correct for an S-meter measurement.

6.4 What This Means for Waterfall and Spectrum Display

The waterfall and spectrum display are computed from the full wideband stream using an FFT. On a MacBook, the vDSP framework computes a 4096-point FFT from the 2 MHz stream in microseconds. The result is a spectrum display covering the full 2 MHz of captured bandwidth, updated at whatever frame rate the client configures (typically 10–30 frames per second). Slice receivers are shown as overlays on this spectrum.

The hardware sends no FFT data. The waterfall is computed entirely in the client from the same raw I/Q stream that feeds the slice receivers. A single ADC output feeds both the spectrum display and all the slice demodulators simultaneously.

Figure 1 — SDR-SLC Slice Model: Host-Side DSP Pipeline



7. System Architecture

7.1 Protocol Planes

An SDR-SLC system has two primary endpoints: the SDR-SLC Device and the SDR-SLC Client. The three protocol planes operate simultaneously and independently:

Plane	Protocol	Transport	Default port
Discovery	SLC-DISC (mDNS/DNS-SD)	UDP multicast 224.0.0.251	5353
Control	SLC-CP (JSON/TCP)	TCP unicast	4620
Data	VITA-SLC (VITA-49)	UDP unicast	Configured via SLC-CP
Time sync	PTPv2 (IEEE 1588)	UDP multicast / L2	319 / 320

7.2 Startup Dependency Order

The planes have a strict dependency order. The client must complete each phase before beginning the next:

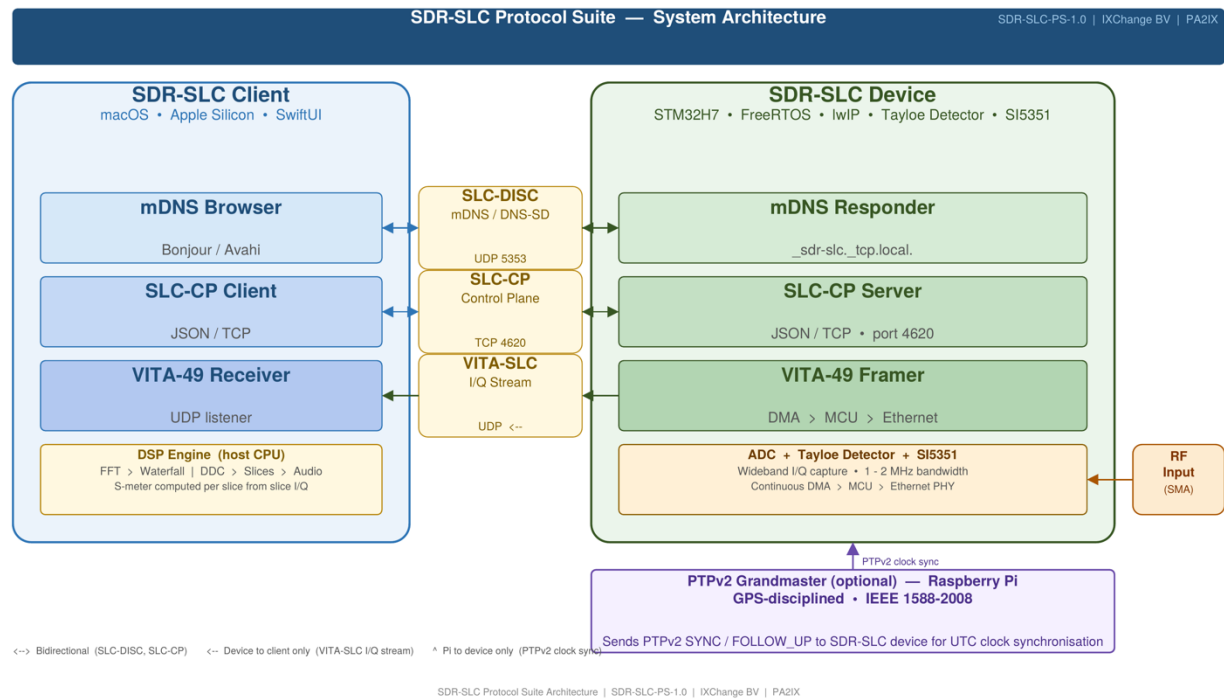
- SLC-DISC first. Browse for `_sdr-slc._tcp.local.`, resolve hostname, read TXT record for device identity and frequency range.
- SLC-CP second. Connect TCP, negotiate protocol version, query capabilities, configure frequency and gain, open stream session.
- VITA-SLC third. After `start_rx`, UDP packets begin flowing to the destination configured in `open_stream`.

TIP

PTPv2 operates independently and in parallel. The device listens for PTP SYNC messages from a grandmaster clock and disciplines its internal RTC automatically. This happens in the background and does not block any other protocol activity. The client reads clock status via SLC-CP `get_clock_status` at any time.

7.3 System Architecture Diagram

Figure 2 — SDR-SLC System Overview



8. Network Topology

8.1 Local Network Operation

All SDR-SLC protocols are designed for operation on a local area network. mDNS is link-local and cannot cross IP routers without special configuration. SLC-CP and VITA-SLC use unicast TCP and UDP that can technically cross routers, but latency-sensitive I/Q streaming is strongly recommended on the same LAN segment.

The typical home installation is a MacBook and one or more SDR-SLC devices connected to the same home router or switch. No configuration is required beyond plugging in the Ethernet cable. The device announces itself via mDNS; the client finds it automatically.

8.2 IP Addressing

Devices may use static or DHCP-assigned IPv4 addresses. SLC-DISC resolves the device hostname (e.g. SLC-000012.local.) to whatever IP address the device currently holds, so the client is never affected by IP address changes. Static addresses are not required but are recommended for repeatable remote access configuration.

8.3 Multiple Devices

Multiple SDR-SLC devices on the same LAN are fully supported. Each device has its own mDNS advertisement with a unique hostname derived from its MAC address. Clients may connect to multiple devices simultaneously, each over an independent TCP control connection and each with its own VITA-49 UDP stream. Resource allocation between multiple simultaneous streams is a client implementation concern; see §6 for the architectural rationale.

8.4 Managed Switches and IGMP

On managed switches with IGMP snooping, the device must send an IGMPv2 Membership Report for multicast group 224.0.0.251 on Ethernet link-up, before the mDNS announcement. Without this report, the switch will not forward mDNS multicast to the device port and discovery will fail.

NOTE

If your device is not discovered on a managed switch, this is almost always the cause. Verify that IGMPv2 Membership Reports are being sent by capturing traffic on the switch port. On an STM32 with lwIP, set `LWIP_IGMP=1` in `lwipopts.h` and call `igmp_joingroup()` for 224.0.0.251 after the network interface comes up.

9. Time Synchronisation

9.1 Role of PTPv2

VITA-SLC I/Q packets carry a UTC timestamp with picosecond fractional resolution. The quality of this timestamp depends on the device's clock source. For most home station use — basic reception, waterfall display, audio demodulation — the absolute accuracy of the timestamp does not matter. The fractional timestamp derived from the sample counter is always accurate relative to the start of the stream, regardless of the absolute clock.

Where absolute timestamp accuracy matters is frequency calibration (comparing your received frequency to a reference standard), time-domain analysis (measuring propagation delays), and multi-receiver correlation (combining signals from geographically separated receivers). For these applications, PTPv2 synchronisation to a GPS-disciplined grandmaster provides sub-microsecond UTC accuracy.

9.2 Clock Source Hierarchy

Source	Accuracy	Notes
GPS PTPv2 grandmaster	< 1 μ s	Best. UTC-traceable. For frequency calibration and multi-RX use.
Non-GPS PTPv2 grandmaster	< 100 μ s	Good for most purposes.
On-device GPS module	< 1 μ s	Good. Requires optional GPS module on the hardware board.
Manual (SLC-CP set_clock)	Client accuracy	Acceptable for casual logging.
Free-running counter	~1 ms/day drift	Relative timestamps only. Sufficient for basic reception.

TIP

For a first build, free-running timestamps are perfectly acceptable. The client can display and demodulate correctly with relative timestamps. Add PTP support when you need frequency calibration or multi-receiver operation. On STM32, small PTP slave implementations are available on [GitHub](#).

10. I/Q Format and Sample Depth

The VITA-49 streaming profile supports per-stream format negotiation. The client reads the available formats from capabilities.rx.iq_formats and selects the most appropriate one. The format is confirmed by the device in the open_stream response and remains fixed for the stream lifetime.

Label	Encoding	Bits	Bytes/pair	Notes
u8	Unsigned offset binary	8	2	Native on RTL-SDR. 128=zero, 0=min, 255=max. Lowest bandwidth.
int16	Two's complement	16	4	Native on most 16-bit ADCs. IXChange SDR native. Big-endian.
int24	Two's complement	24	8	High dynamic range ADCs. Left-justified in 32-bit container.
float32	IEEE 754 single	32	8	Normalised to ADC full scale. Reserved for future hardware.

NOTE

All VITA-SLC encodings use big-endian byte order. Samples are interleaved: $I_0 Q_0 I_1 Q_1 \dots$. An MTU-safe packet carries 360 IQ pairs at int16 (1496-byte datagram, 4 bytes under the 1500-byte Ethernet MTU).

11. Conformance

Conformance is defined by four implementation profiles, matched to the SLC-CP specification profiles. A device claims a profile by advertising it in the SLC-DISC TXT record (role field) and implementing the commands for that profile.

Profile	Description	Typical hardware
Tier 1 Minimal RX	Nine commands. Samples flow. Nothing else required.	RTL-SDR daemon on Raspberry Pi; bare STM32 receiver.
Tier 1 Minimal RX/TX	Adds basic TX stream, PTT. No VSWR or auto-tune.	Home-built Tayloe transceiver; QRP digital radio.
Tier 2 Standard	Full control set, AGC, Reference Level, Context packets, events.	Well-engineered RTL-SDR daemon; feature-complete firmware.
Tier 3 Full	VSWR, GPS, PTP, hardware monitors, auto-tune.	IXChange SDR hardware v1; advanced implementations.

11.1 Device Conformance by Profile

Requirement	Protocol	T1 RX	T1 TX	T2	T3
mDNS advertisement (_sdr-slc._tcp.local.)	SLC-DISC	SHALL	SHALL	SHALL	SHALL
Mandatory TXT record fields	SLC-DISC	SHALL	SHALL	SHALL	SHALL
IGMPv2 Membership Report on link-up	SLC-DISC	SHALL	SHALL	SHALL	SHALL
mDNS goodbye on clean shutdown	SLC-DISC	SHOULD	SHOULD	SHOULD	SHALL
TCP framing + JSON envelope	SLC-CP	SHALL	SHALL	SHALL	SHALL
hello, get_capabilities (minimal)	SLC-CP	SHALL	SHALL	SHALL	SHALL
set_frequency, set_sample_rate, set_control(rf_gain)	SLC-CP	SHALL	SHALL	SHALL	SHALL
open_stream(rx), start_rx, stop_rx, close_stream	SLC-CP	SHALL	SHALL	SHALL	SHALL
TX commands (open_stream tx, start_tx, stop_tx)	SLC-CP	Omit	SHALL	SHALL	SHALL
ping, get_status (full)	SLC-CP	MAY	MAY	SHALL	SHALL
Standard controls (agc, bias_tee, ppm, direct_sampling)	SLC-CP	MAY	MAY	SHALL	SHALL
Reference Level (capabilities + get_status)	SLC-CP	MAY	MAY	SHALL	SHALL
get_clock_status	SLC-CP	MAY	MAY	SHOULD	SHALL
get_vswr, auto_tune	SLC-CP	Omit	Omit	Omit	SHALL
hardware_monitors, monitor_alert	SLC-CP	Omit	Omit	SHALL	SHALL
IF Data packets (VITA-A/1.0)	VITA-SLC	SHALL	SHALL	SHALL	SHALL

Requirement	Protocol	T1 RX	T1 TX	T2	T3
Context packets (VITA-A/1.0-CTX)	VITA-SLC	MAY	MAY	SHALL	SHALL
PTPv2 clock discipline	PTPv2	MAY	MAY	SHOULD	SHALL

11.2 Client Conformance

Requirement	Level
Browse _sdr-slc._tcp.local. before connecting	SHALL
Parse minhz and maxhz as 64-bit unsigned integers	SHALL
Send hello as first message; verify selected protocol version	SHALL
Call get_capabilities before any radio command	SHALL
Treat all _hz fields as 64-bit unsigned integers	SHALL
Silently ignore unknown JSON fields and unknown events	SHALL
Silently discard VITA-49 Packet Type 0x4 if Context parsing not implemented	SHALL
Not infer data loss from Context packet Packet Count	SHALL
Handle unknown_command and not_supported gracefully (disable UI element, no error)	SHALL
Re-execute full startup sequence after TCP disconnection	SHALL
Use actual_sample_rate_sps from get_status for frequency axis calculations	SHOULD
Treat VITA-49 Context packet Reference Level as authoritative	SHOULD

12. Typical Session Flow

Figure 2 — Complete RX session from device boot to stream active

```

DEVICE BOOT
STM32 initialises: Ethernet link up, DHCP, IGMP join 224.0.0.251
mDNS announcement: SLC-000012. sdr-slc. tcp.local. port 4620
PTPv2 SYNC received (if grandmaster present) → clock slews to UTC

DISCOVERY (client)
Browses_sdr-slc. tcp.local.
Resolves_SLC-000012.local. → 192.168.5.35
Reads TXT: proto=sdr-slc, fw=1.0.0, role=rxtx, minhz=1800000, maxhz=30000000

CONTROL HANDSHAKE
C→S hello(["1.0"])
S→C device="IXChange SDR V1.0", selected=1.0
C→S get capabilities
S→C [preferred rate, preferred gain, Reference Level, controls, formats]
C→S get_clock_status → verify sync state

CONFIGURATION
C→S set_frequency(rx, 14100000)
C→S set_sample_rate(rx, 1024000) // preferred_sample_rate_sps
C→S set_control(rx, rf_gain, 20.0) // preferred_gain db

STREAM SETUP
C→S open_stream(rx, vita49, VITA-A/1.0, dest=192.168.1.20:60000)
S→C stream_id=rx-1, src=192.168.5.35:4621, samples_per_packet=360
C→S start_rx(rx-1)
S→C ok + event stream_state(rx-1, streaming=true, direction=rx)

ACTIVE RECEPTION
[UDP] Context packet: RL=-30.0dBm, complex IQ
[UDP] IF Data packets: 360 IQ pairs, ~5689 pkt/s at 1 MSPS
Client: FFT → waterfall; DDC → slice demodulation → audio
Client: S-meter from slice I/Q amplitude + Reference Level

SHUTDOWN
C→S stop_rx(rx-1)
C→S close_stream(rx-1)
TCP close

```

13. Design Rationale

13.1 Why mDNS/DNS-SD?

mDNS is natively supported on macOS (Bonjour), Linux (Avahi), and Windows 10+. No server infrastructure, no configuration, no IP addresses to remember. Plug in the cable and the client finds the device. DNS-SD TXT records carry structured metadata — frequency range, firmware version, device role — readable before the TCP connection is opened.

13.2 Why JSON/TCP?

JSON is human-readable and debuggable with standard tools (netcat, Wireshark, curl). You can test every command by hand before writing a single line of client code. TCP provides ordered, reliable delivery for a stateful control protocol. Length-prefixed framing is trivial to implement on STM32 with lwIP. Binary protocols would save a few hundred bytes per message at the cost of debuggability — the wrong trade-off for an open community project.

13.3 Why VITA-49?

VITA-49 is an open, documented standard for I/Q transport with built-in timestamping, stream identification, and in-band metadata via Context packets. Using an established standard means the packet format is documented independently of this project, existing VITA-49 analysis tools work on SDR-SLC streams, and the IXChange BV OUI in the Class Identifier provides globally unique packet identification without any central registry.

13.4 Why Host-Side DSP?

Because the host is incomparably more powerful than any hardware that fits in a €100 budget, and it is already there. A MacBook with Apple Silicon runs more signal processing than any FPGA at any price point the amateur market would accept. Putting DSP on the host also means algorithms are in software: updating, improving, or replacing demodulation code does not require a hardware change or a firmware update from the manufacturer. The experimenter has full access to the DSP.

13.5 Why 64-bit Frequency Fields?

32-bit unsigned integers represent frequencies up to 4.295 GHz. Microwave SDR hardware operates above this limit. Using 64-bit integers in JSON costs nothing on the wire and ensures the protocol never artificially limits future hardware. On STM32, use `uint64_t`. On JavaScript clients, use `BigInt`.

14. Hardware Reference Design

This section provides guidance for HAM operators building their own SDR-SLC hardware. It is informative, not normative. Any hardware that correctly implements the protocol is a conformant SDR-SLC device regardless of its internal construction.

14.1 The Tayloe Detector

The quadrature sampling detector invented by Dan Tayloe (N7VE) is the preferred RF front end for SDR-SLC homebrew hardware. It consists of four bilateral CMOS switches (e.g. FST3253, 74HC4066) driven by a quadrature clock at four times the desired receive frequency, feeding two differential integrators. The output is a direct I/Q baseband signal at a fraction of the component count of a conventional superhet front end.

The Tayloe detector has excellent dynamic range for HF use, typically 100+ dB if the op-amp stage is carefully chosen (NE5532, OPA2134, or better). It requires no IF filter and produces zero IF. The SI5351A clock synthesiser, controlled over I2C from the microcontroller, provides the quadrature drive clock at 4x the receive frequency, covering the full HF range from 1.8 MHz to 30 MHz.

TIP

The FST3253 is preferred over the 74HC4066 for the bilateral switches. It has lower on-resistance and better high-frequency performance, which gives a cleaner image rejection. The NE5532 is a cost-effective op-amp choice; the OPA2134 gives noticeably lower noise floor if you are building for weak-signal work. Both are in DIL packages and easy to work with on veroboard.

14.2 Microcontroller

The STM32H743 or H745 (dual-core) is the recommended microcontroller. The H7 series includes a hardware Ethernet MAC, DMA for continuous peripheral transfers, hardware SPI for the ADC interface, a hardware timer for packet pacing, and sufficient SRAM (1 MB) for lwIP TCP/IP buffers, VITA-49 packet assembly buffers, and the FreeRTOS task stack. The H7 is available in LQFP-144 packages that are solderable by hand with practice.

Lower-cost alternatives include the STM32F407 (no hardware Ethernet MAC, requires an SPI Ethernet module such as W5500) and the STM32H563 (newer, lower cost than H7 but same Ethernet capability). For a Raspberry Pi-based implementation, the Pi 4 or Pi 5 running Linux with an RTL-SDR daemon is a conformant Tier 1 device without any custom hardware.

14.3 ADC Interface

The STM32H7 has a built-in 16-bit ADC, but its maximum sample rate in free-running mode (approximately 3.6 MSPS for single-channel) comes with caveats around input impedance and noise. For best performance, an external sigma-delta ADC with SPI output (such as the AD7380 or ADS131E04) driven from a precise clock gives better SNR and cleaner sample timing. The ADS131E04 provides four simultaneous channels at up to 64 kSPS each, which is more than sufficient for HF I/Q capture at the bandwidths SDR-SLC targets.

For a simple first build, the STM32H7 internal ADC in dual interleaved mode can achieve 1 MSPS with acceptable performance. Start with the internal ADC to validate the firmware, then upgrade to an external ADC when optimising.

14.4 A Minimal Working Build

The following component list represents the minimum for a working Tier 1 RX device:

- STM32H743 NUCLEO-H743ZI development board (€25–30). Includes Ethernet connector, USB for programming, and all support circuitry.
- SI5351A breakout board (€5). Provides quadrature clock via I2C.
- 74HC4066 quad bilateral switch or FST3253 (€1–2). Tayloe detector switches.
- 2× NE5532 dual op-amp (€1–2). I and Q differential amplifiers.
- Band-pass filter for the desired band. A simple LC ladder or a commercial plug-in module (€5–10).
- Passive components (resistors, capacitors) from the junk box.
- SMA connectors for RF input/output.

Total cost for a NUCLEO-based prototype: approximately €35–45. This runs the full SLC-CP protocol and streams I/Q over Ethernet to the Swift client. Once the firmware is validated, move to a custom PCB for a more permanent installation.

TIP

The STM32 NUCLEO boards are an excellent starting point because they handle the power supply, USB interface, and crystal oscillator, leaving you free to focus on the RF circuitry and firmware logic. The Ethernet connector on the NUCLEO-H743ZI is a standard RJ45; just plug it into your home switch.

15. Security Considerations

SDR-SLC version 1.0 is designed for trusted LAN operation. No authentication, authorisation, or encryption is provided at the protocol level. Any host on the LAN that can reach TCP port 4620 can issue all control commands including setting the transmit frequency and starting transmission.

WARNING

*On any installation where untrusted hosts share the LAN, use network-level access controls (firewall rules, VLAN isolation) to restrict access to the SDR-SLC control port. On an Internet-connected installation, the control port **MUST** be firewalled from the Internet. The transmit commands are particularly sensitive: any authenticated user can key up your station.*

mDNS advertisements are unauthenticated. A rogue host could advertise a fake SDR-SLC device and redirect clients. Clients **SHOULD** verify the manufacturer and proto TXT record fields before connecting. A future protocol version **MAY** introduce TLS for the TCP control connection.

16. Forward Compatibility

Scenario	Behaviour
Unknown JSON fields	Silently ignored by all receivers.
Unknown SLC-CP commands	Return unknown_command. Connection stays open.
Unknown events	Silently ignored by clients.
Unknown VITA-49 packet types	Silently discarded.
Unknown CIF0 bits in Context packet	Skip the corresponding field bytes; process others.
New iq_formats entries	Added with supported:false until implemented.
Minor version (1.0 → 1.1)	Backward compatible. Adds optional fields and commands.
Major version (1.x → 2.0)	Breaking. Requires new hello version negotiation.

17. References

17.1 Normative

Reference	Title	Relevance
SDR-SLC-DISC-1.0	SLC-DISC Service Discovery	Device discovery; provides IP and port.
SDR-SLC-CP-1.0	SLC-CP Control Plane	JSON/TCP radio control protocol.
SDR-SLC-VITA-1.0	VITA-SLC Stream Transport	VITA-49 I/Q packet format.
RFC 6762	Multicast DNS	mDNS protocol.
RFC 6763	DNS-Based Service Discovery	DNS-SD protocol.
RFC 2119	Key Words for Use in RFCs	Normative keywords.
IEEE 1588-2008	PTPv2	Clock synchronisation.
VITA-49.0	VITA Radio Transport Standard	Base VRT packet format.
RFC 768	User Datagram Protocol	UDP transport for VITA-SLC.
RFC 793	Transmission Control Protocol	TCP transport for SLC-CP.

17.2 Informative

Reference	Title	Notes
IEEE 802 / OUI	OUI Registry	IXChange BV OUI 40-28-14.
RFC 4627	JSON Specification	JSON encoding.
FreeRTOS	FreeRTOS RTOS	Recommended RTOS for STM32 implementations.
lwIP	Lightweight IP Stack	Recommended TCP/IP stack for STM32.
cJSON	Ultralight JSON Parser	Recommended JSON library for embedded targets.
Tayloe 2001	QEX July/August 2001 — N7VE	Original Tayloe detector description. Required reading for builders.

Appendix A Document Identifier Register

Identifier	Type	Value	Description
IXChange BV OUI	IEEE MA-L OUI	40-28-14 (0x402814)	Used in VITA-49 Class Identifier and SLC-DISC TXT record.
VITA-A/1.0 PCC	VITA-49 PCC	0x0001	IF Data packet class for SDR-SLC streams.
VITA-A/1.0-CTX PCC	VITA-49 PCC	0x0001	Context packets share PCC with IF Data; distinguished by Packet Type.
_sdr-slc._tcp	DNS-SD service type	—	Service browse type for SLC-DISC.
SLC-XXYYZZ	mDNS hostname prefix	Per device	XX YY ZZ = last 3 octets of MAC address, uppercase hex.